Solution: Number of Substrings Containing All Three Characters

Problem Statement

Given a string `s` containing only `a`, `b`, and `c` characters, return the number of substrings containing `at least` one occurences of `a`, `b` and `c` characters.

Examples

- **Example 1**:
  - **Input**: `"abc"`
  - **Expected Output**: `1`
  - **Justification**: The string itself ("abc") is the only substring containing all three characters.
- **Example 2**:
  - **Input**: `"abbcca"`
  - **Expected Output**: `5`
  - **Justification**: The substrings containing all 3 characters are "abbc", "abbcc", "abbcca", "bbcca", and "bcca".
- **Example 3**:
  - **Input**: `"abababacb"`
  - **Expected Output**: `9`
  - **Justification**: The substrings containing all 3 characters are "ababac", "babac", "abac", "bac", "ababacb", "babacb", "abacb", "bacb", and "acb".

Solution

To solve this problem, we will use a sliding window approach. This approach is effective as it allows us to dynamically adjust the size of our window (substring) to include all three characters, 'a', 'b', and 'c', while minimizing unnecessary repetitions. The crux of this method is to expand and contract the window based on the presence of 'a', 'b', and 'c' in it. When the window contains all three characters, we'll calculate the number of new substrings this window adds to our total count. This approach is efficient because it requires only one pass through the string, and it's easy to understand and implement.

Step-by-Step Algorithm

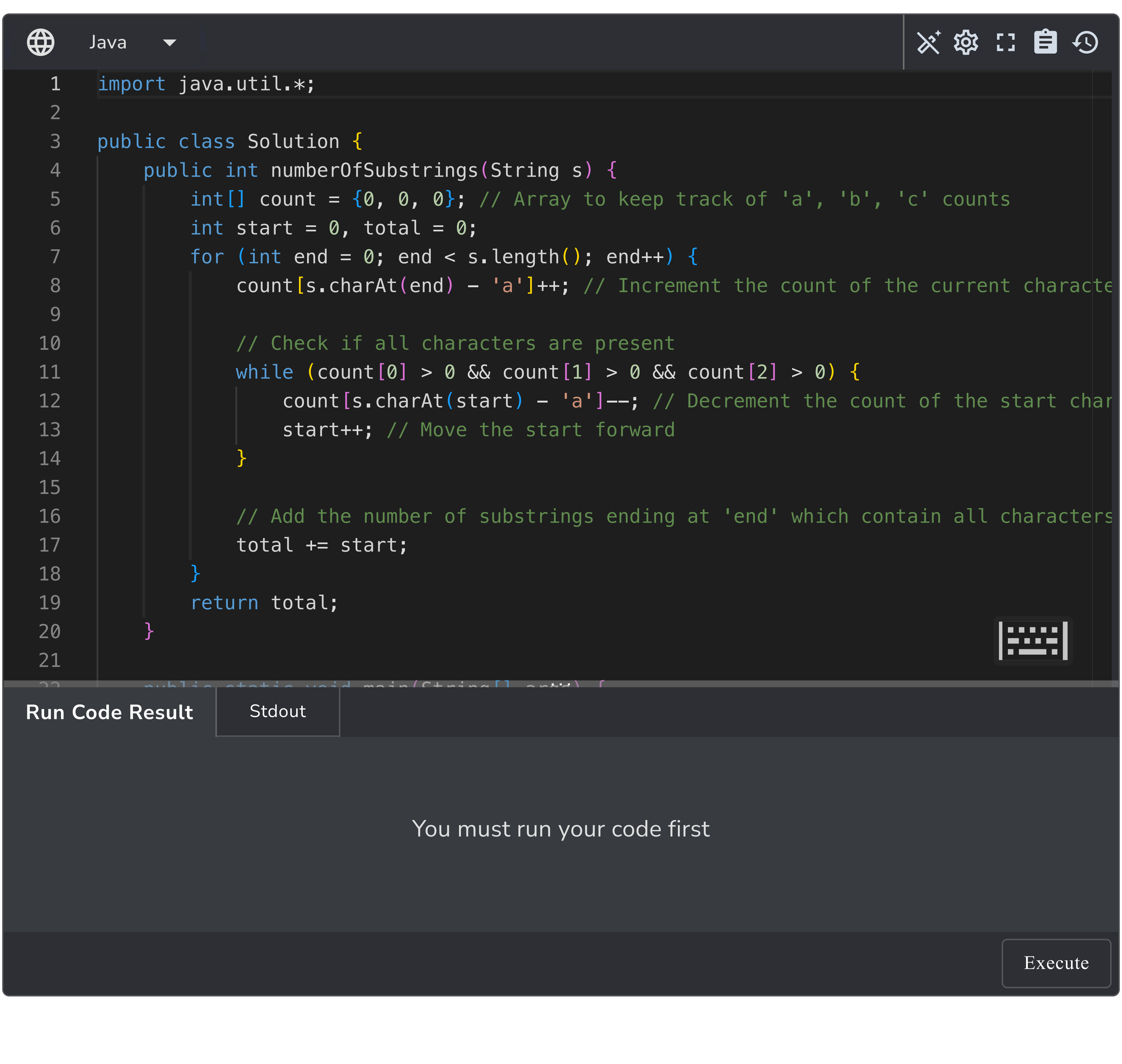
1. **Initialize Count Array and Pointers**:
   - Create an array `count` with 3 elements (for 'a', 'b', 'c'), all initialized to 0.
   - Initialize two pointers: `start` at 0 and `end` at 0. Initialize `total` to 0 for counting valid substrings.
2. **Iterate with `end` Pointer**:
   - Incrementally move the `end` pointer from the start to the end of the string.
   - For each character at `end`, increase its corresponding count in the `count` array.
3. **Check and Adjust `start` Pointer**:
   - Inside the loop, check if all three characters are present (each count > 0).
   - If so, move the `start` pointer forward, decrementing the count of the character at the `start` position. Repeat until the window (from `start` to `end`) no longer contains all three characters.
4. **Update Total Count**:
   - After adjusting the `start` pointer, add `start` to `total`. This accounts for all valid substrings ending at the current `end` character.
   - Continue until `end` has covered the entire string.
5. **Return Total Count**:
   - Return `total` as the number of substrings containing all three characters.

Algorithm Walkthrough

Let's consider the input: `ababacb`.

- **Initialize**: `count = [0, 0, 0]`, `start = 0`, `end = 0`, `total = 0`.
- **Iteration**:
  - `end = 0` (`'a'`): Increment `count[0]`. `count = [1, 0, 0]`. No valid substring yet.
  - `end = 1` (`'b'`): Increment `count[1]`. `count = [1, 1, 0]`. No valid substring yet.
  - `end = 2` (`'a'`): Increment `count[0]`. `count = [2, 1, 0]`. No valid substring yet.
  - `end = 3` (`'b'`): Increment `count[1]`. `count = [2, 2, 0]`. No valid substring yet.
  - `end = 4` (`'a'`): Increment `count[0]`. `count = [3, 2, 0]`. No valid substring yet.
  - `end = 5` (`'c'`): Increment `count[2]`. Now all characters are present (`count = [3, 2, 1]`).
    - Start moving `start` while the window is valid (all counts > 0):
      - `start = 0`: Decrement `count[0]` (character at `s[0]` is 'a'). `count = [2, 2, 1]`.
      - `start = 1`: Decrement `count[1]` (character at `s[1]` is 'b'). `count = [2, 1, 1]`.
      - `start = 2`: Decrement `count[0]` (character at `s[2]` is 'a'). `count = [1, 1, 1]`.
      - `start = 3`: Decrement `count[1]` (character at `s[3]` is 'b'). `count = [1, 0, 1]`.
      - `start = 4`: Stop as 'b' is no longer in the window.
        `total = total + start = 0 + 4 = 4`.
  - `end = 6` (`'b'`): Increment `count[1]`. `count = [1, 1, 1]`.
    - Start moving `start` again:
      - `start = 4`: Decrement `count[0]`. `count = [0, 1, 1]`.
      - `start = 5`: Stop as 'a' is no longer in the window.
        `total = total + start = 4 + 5 = 9`. Total substrings = `9`.
- **Final Total**: `total = 9`.

Code



```java
import java.util.*;

public class Solution {
    public int numberOfSubstrings(String s) {
        int[] count = {0, 0, 0}; // Array to keep track of 'a', 'b', 'c' counts
        int start = 0, total = 0;
        for (int end = 0; end < s.length(); end++) {
            count[s.charAt(end) - 'a']++; // Increment the count of the current characte

            // Check if all characters are present
            while (count[0] > 0 && count[1] > 0 && count[2] > 0) {
                count[s.charAt(start) - 'a']--; // Decrement the count of the start char
                start++; // Move the start forward
            }

            // Add the number of substrings ending at 'end' which contain all characters
            total += start;
        }
        return total;
    }

```

Complexity Analysis

- **Time Complexity**: O(N)
  - Each character in the string is visited at most twice (once by the `end` pointer and once by the `start` pointer).
  - Despite having nested loops, the total number of operations remains linear with the length of the string.
- **Space Complexity**: O(1)
  - The space used does not scale with the size of the input.
  - A fixed-size array is used to keep track of the counts of 'a', 'b', and 'c', leading to constant space usage.

← Previous
Number Of Substrings Containing All Three Characters (Medium)

Next →
Capacity To Ship Packages Within D Days (Medium)

☐ Mark as Completed